Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.1.1 sur 153 | difficulté 1 / 10

0.1.1. Afficher la lettre A — de base, ailleurs

Le même A qu'au 0.1, mais au milieu de l'écran : texte(10, 8, "A").

1. Ce qu'il faut comprendre

C'est le 0.1, avec un seul changement : la **place** du A. Il n'est plus en (0, 0), en haut à gauche, mais en **(10, 8)**, au milieu de l'écran.

Les deux premiers nombres de `texte` sont la colonne (de 0 à 19, de gauche à droite) et la ligne (de 0 à 17, de haut en bas). Le milieu de l'écran est vers la colonne 10 et la ligne 8.

C'est la version de base : une seule chose, à la deuxième place. Le 0.1.2 met les deux ensemble.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <texte>` → 0.0.2. La fonction texte() — écrire un mot

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **chaîne de caractères (texte)** — Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **texte()** — Écrit des lettres à l'écran, sur la grille de 20 colonnes et 18 lignes : texte(colonne, ligne, "MOT").
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <texte>   // écrit un texte à l'écran`	Demande à la console « texte » (ce que c'est : écrit un texte à l'écran, à la taille qu'on veut). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un texte à l'écran ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`// Le changement : la place. (0, 0) → (10, 8).`	Un commentaire : une phrase pour celui qui lit (« Le changement : la place. (0, 0) → (10, 8). »). La console l'ignore.
`//   texte(10, 8, "A");`	Un commentaire : une phrase pour celui qui lit (« texte(10, 8, "A"); »). La console l'ignore.

`//        |   |`	Un commentaire : une phrase pour celui qui lit (« | | »). La console l'ignore.
`//        |   +-- ligne 8   : vers le milieu, de haut en bas (0 à 17)`	Un commentaire : une phrase pour celui qui lit (« | +-- ligne 8 : vers le milieu, de haut en bas (0 à 17) »). La console l'ignore.
`//        +------ colonne 10 : vers le milieu, de gauche à droite (0 à 19)`	Un commentaire : une phrase pour celui qui lit (« +------ colonne 10 : vers le milieu, de gauche à droite (0 à 19) »). La console l'ignore.
`texte(10, 8, "A");`	Appel de la fonction texte de la console, avec les arguments 10, 8, "A". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **203 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  main()           18 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EcrireTexte  — attend de pouvoir écrire une case
  EcrireTexte      10 o  ← texte()  — écrit un texte, lettre par lettre
  CopierTuiles     31 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles           16 o  ← CopierTuiles  — 1 lettre sur 43
  "A"               1 o  ← texte()  — un texte gravé
Lettres gravées : A
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  texte() × 1    11 o sur place, dans main()  + EcrireTexte (10 o)
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texte>   // écrit un texte à l'écran
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

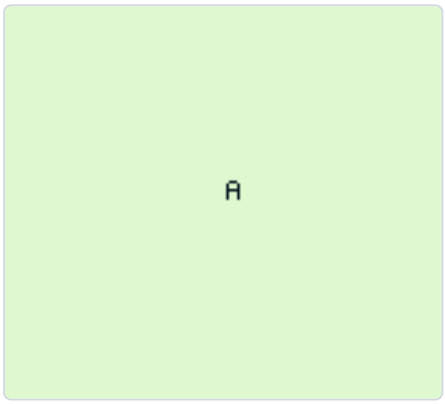
```
int main() {
  // Le changement : la place. (0, 0) → (10, 8).
  //
  //   texte(10, 8, "A");
  //         |   |
  //         |   +-- ligne 8   : vers le milieu, de haut en bas (0 à 17)
  //         +------ colonne 10 : vers le milieu, de gauche à droite (0 à 19)
  texte(10, 8, "A");

  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> Un A au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A est au milieu, en (10, 8)

✓ et plus en haut à gauche

Ensuite : **0.1.2. Afficher la lettre A — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++